FUXA — Codebase and Project Architecture

Web-based Process Visualization (SCADA / HMI / Dashboard)

Repository: frangoteam/FUXA v1.3.5-2937 — deployment with the iPTran Leak Detection HMI

Generated 15 October 2026 from the working tree at /home/bfa/fuxahmi/FUXA

Scope: complete repository structure, server runtime, client application, data model, widget system, deployment and CI

1. Executive Summary

FUXA is an open-source, full-stack SCADA/HMI platform. A **Node.js/Express server** acquires live process data from industrial devices (Modbus, OPC UA, S7, BACnet, MQTT and 13 more), historizes tag values into SQLite/InfluxDB/ QuestDB/TDengine, exposes a secured REST API over HTTP, and pushes live values to a rich **Angular 18 single-page client** over Socket.IO/ WebSocket. The client provides a drawing-style **Editor** in which operators compose SVG views, bind device tags to graphic elements and widgets, and configure alarms, notifications, schedulers, reports and recipes. The finished project is served to plant operators as a fullscreen HMI (Home), a simulation **Lab**, a kiosk mode, an Electron desktop app, and — in this fork — a browser-based **AR mode** and a custom **iPTran leak-detection schematic widget**.

Dimension	Facts
Codebase size	server: ~134 JS modules (excl. node_modules/dist), client: ~182 Angular components (client/src ~15 MB)
Server stack	Node.js 18, Express 4, Socket.IO 4, sqlite3 5, InfluxDB clients, Node-RED 4 (optional), Winston logger, Mocha/Sinon tests
Client stack	Angular 18 + Angular Material, RxJS, SVG.js/Raphael/Snap for gauges, ngx-translate, gridster, CodeMirror
Packaging	Multi-stage Docker image (client build → server build → runtime), docker compose, Electron app, headless runtime
Documentation	MkDocs (Material) site in docs/ (Python toolchain, see requirements.txt)
CI	GitHub Actions: docker image builds (main/rc/release), docs deploy, Electron builds, headless packaging

2. Repository Top Level

```
FUXA/
├── compose.yml            Docker Compose for this deployment (bind mounts incl. ./widgets)
├── Dockerfile             Multi-stage production image (client build → server build →
runtime)
├── runbook.md             Operational runbook for this deployment (start → configure → bind
→ test → stop)
├── requirements.txt       Python deps: MkDocs + Material (documentation site only)
├── mkdocs.yml             Docs site configuration (nav, Material theme)
├── LICENSE, README.md, CONTRIBUTING.md, SECURITY.md
├── docs/                  Operator/developer documentation (~30 HowTo pages, ar-mode.md
design note)
├── server/                Node.js backend + REST API + device drivers + DAQ storage (see §3)
├── client/                Angular 18 editor/HMI frontend (see §4); client/dist = prebuilt
bundle
├── app/                   Desktop & headless packaging roots
│   ├── electron/          Electron wrapper (main.js, settings UI, project selection, icons)
│   └── headless/          headless-entry.js — server-only runtime without editor
├── node-red/              node-red-contrib-fuxa — Node-RED palette nodes that talk to FUXA
├── screenshot/            Marketing screenshots
└── .github/workflows/     CI pipelines (docker_image, docker_rc, docker_release, docs,
electron_latest, headless_packaging)
```

2.1 Docker deployment layout (this machine)

The running instance is the stock `frangoteam/fuxa:latest` image with five host bind mounts; all project state survives container removal:

Host path	Container path	Contents
FUXA/appdata	server/_appdata	settings.js, mysettings.json, project.fuxap.db, users/alarms/apikeys/recipes/scheduler DBs, _upload_files
FUXA/db	server/_db	DAQ history (per-device SQLite files + archive/), currentTagReadings.db
FUXA/logs	server/_logs	Rotating server logs + api.log (Morgan)
FUXA/images	server/_images	Image resources (Pipes, Tanks shape libraries)
FUXA/widgets	server/_widgets	Widget library served to the editor (dynamicSVG group)

3. Server (server/)

3.1 Boot sequence — main.js (683 lines)

1. Parse CLI args (`nopt`): `--port`, `--userDir`, `--env`; env `userDir`, `BASE_PATH`.
2. Resolve workDir (`<userDir>/_appdata`), copy `settings.default.js` → `_appdata/settings.js` on first run, merge missing fields when the version differs (deepMerge).
3. Read user overrides `_appdata/mysettings.json` (language, uiPort, secure settings, smtp, daqstore, alarms, logs, nodeRed, swagger...).
4. Generate a random JWT secret if `secureEnabled` without `secretCode` (fail-safe).
5. Create runtime directories: _appdata, _db, _reports, _upload_files, _images, _widgets, _webcam_snapshots, _logs.
6. Create the HTTP(S) server with Express + Socket.IO (path = BASE_PATH + /socket.io, EIO3 compat).
7. Optional `BASE_PATH` reverse-proxy support: rewrites <base href> and asset URLs in memory.
8. Static routes: Angular shell routes (/, /home, /editor, /lab, /device, /view, /ar, ...), `/resources`, `/_images`, `/_widgets`, webcam snapshots (token-protected).
9. Morgan request logging to `_logs/api.log` (level from settings), optional Swagger UI at /api-docs.
10. Call `FUXA.init()` (fuxa.js) which initializes the runtime (§3.2) and mounts the API; on `init-runtime-ok` (or a 60 s timeout) start listening on uiPort (default 1881).
11. Webcam snapshot cleanup scheduled daily at 01:00 (node-schedule) when enabled.
12. SIGINT handler stops the runtime cleanly; uncaughtException logger keeps the process alive.

3.2 Runtime core — server/runtime/

Module	Responsibility
index.js	Runtime orchestrator: initializes and exposes every subsystem (devices, daqstorage, project, alarms, users, notifications, scheduler, scripts, plugins); handles Socket.IO subscriptions (viewsTagsSubscribe / tag value broadcast), and periodic retention checks.
events.js	Minimal internal pub/sub used between subsystems (init-runtime-ok, access:login, tag changes).
logger.js	Winston wrapper with per-run log file under _logs.
utils.js	Shared helpers (deepMerge, retention windows, date formatting, guards).
project/	Project persistence: prjstorage.js (project.fuxap.db with tables general/views/devices/devicesSecurity/texts/alarms/recipes/notifications/scripts/reports/locations/arMarkers), textstorage for multi-language texts.
users/	usrstorage.js (users.fuxap.db, bcrypt-hashed passwords, roles/groups), token helper.
apikeys/	apiKeysStorage.js — REST API keys for machine-to-machine access.
alarms/	alarmstorage.js + alarm runtime: threshold/derived alarms from device tags, ack workflow, alarm history.
notificator/	Alarm/notify storage and e-mail delivery via nodemailer (SMTP from settings).
recipes/	Recipe storage and serving (batch recipes bound to view controls).

scheduler/	Schedule storage; node-schedule jobs trigger scripts/reports.
scripts/	User JS scripts storage and sandboxed execution (run on schedule/event).
jobs/	Report generation engine: report.js (pdfmake with custom fonts in jobs/fonts), cleaner.js, helper/.
plugins/	Plugin manager (§3.5): npm runtime installer, plugin registry, device driver swapping.

3.3 Device layer — server/runtime/devices/

device.js implements a per-device state machine (INIT/IDLE, connect check every 5 s, polling interval default 3000 ms) and dispatches to a communication client by device type. Each driver module exports `create(data, logger, events, pluginManager, runtime)` and implements the client contract: connect, disconnect, isConnected, polling, load, getValue(s), setValue, bindAddDaq, getTagProperty, getStatus, lastReadTimestamp (+ optional browse for address discovery).

Driver folder	Protocol / type	Notes
modbus/	ModbusRTU / ModbusTCP	modbus-serial; class 0/1 read, writes
opcua/	OPC UA	node-opcua; address-space browse, security policies
s7/	SiemensS7	node-snap7 (optional native plugin)
bacnet/	BACnet	node-bacnet
mqtt/	MQTTclient	mqtt.js; topic browse, QoS, TLS
httprequest/	WebAPI	REST polling (GET/POST), axios
ethernetip/	EthernetIP	nodepccc (Allen-Bradley)
omron-ethernetip/	OmronEthernetIP	omron-ethernet-ip
melsec/	MELSEC	mcprotocol (Mitsubishi)
adsclient/	ADSclient	ads-client (Beckhoff TwinCAT)
odbc/	ODBC	Database device via unixodbc + table browse
redis/	REDIS	Redis keys as tags
fuxaserver/	FuxaServer	Internal simulation device (used by this deployment) and FUXA-to-FUXA bridge
gpio/, webcam/, template/	GPIO, WebCam, (template)	Raspberry GPIO, snapshot capture; template = reference driver skeleton
device-utils.js	—	tagDaqToSave (DAQ change/interval decision), tagValueCompose (scaling scripts)

> **DAQ write path (as built today):** device polling → `addDaq(changedTags, name, id)` (bound in devices/index.js via `daqStorage.addDaqNode`) → storage sink. The SQLite sink inserts one row per tag per poll with a boolean “working” guard and drops batches under load (data-overload warning) — documented in the runbook as the current scalability limit (~1 Hz with DAQ).

3.4 Storage layer — server/runtime/storage/

Module	Role
daqstorage.js	

	DAQ sink factory: per-device SQLite DaqNode, or a single shared sink for InfluxDB (1.8/2.x), TDengine, QuestDB; tag map resolution; retention checks.
sqlite/index.js	Per-device DaqNode: daq-map_<dev>.db (tag map: mapid/id/name/type) + time-sliced daq-data_<dev>_<timestamp>.db (dt/id/value rows); hourly tokenization, archive/ folder rotation, retention deletion.
sqlite/ currentstorage.js	currentTagReadings.db — last value per tag, batched 5 s write queue (used for value restore on device reconnect).
influxdb/, questdb/, tdengine/	External historian sinks selected via settings.daqstore.type (url/organization/ bucket/token).
calculator.js	Chart aggregation functions (min/max/avg/sum, interpolation, formats) over DAQ series.

3.5 Plugin system — server/runtime/plugins/

FUXA ships drivers as npm packages that are installed at runtime into an isolated package directory (`_pkg`): **npm-runtime-service.js** implements install/uninstall/list of npm packages outside the main node_modules; **index.js** keeps a registry (Plugin records: name/module/ type/version/group) with groups `connection-device`, `connection-database`, `chart-report`, `service` (node-red). `device.loadPlugin(type, module)` hot-swaps the driver module for a device type. The editor Plugins page drives this via /api/plugins.

3.6 REST API — server/api/

index.js mounts each group on Express with JWT auth (jwt-helper.js, x-access-token / refresh cookie), role checks and API rate limiting. Groups:

Route group	Purpose
auth/	/api/signin, /api/refresh, logout, token helpers, bcrypt user validation
projects/	Load/save/export project JSON, settings disclosure guard (daqstore credentials never sent to client)
daq/	Chart data queries: aggregated tag values over time ranges (calculator functions)
alarms/	Alarm definitions, active alarms, ack, alarm history
devices (+resources)/	Device CRUD, tags, connection status, browse; resource/image/widget galleries (GET /api/resources/widgets walks _widgets subfolders), uploads, webcam snapshots
users/, apikeys/	User & role management, REST API keys
recipes/, reports/, scheduler/, scripts/	Recipe, report (pdfmake), schedule and script management
command/	Tag write commands (device.setValue)
plugins/	Install/list/remove runtime npm plugins
diagnose/	Server health/diagnostics endpoint

3.7 Integration and packaging

- **integrations/node-red/** — mounts Node-RED (when nodeRedEnabled) on the same HTTP server, with auth mode secure/legacy-open, isolated userDir and the node-red-contrib-fuxa palette nodes.

- **server/dist/reports.service.js** — small TypeScript-compiled helper (tsconfig.json) for report rendering.
- **server/test/** — Mocha + Sinon suites: api, authorization, devices, alarms, storage (QuestDB), notificator, plugins, recipes, help.
- **envParams.js** — optional file injecting DEVICES (device list) from environment for headless boot.
- **_images/** — shared shape libraries (Pipes, Tanks) served statically.

4. Client (client/) — Angular 18 Editor and HMI

4.1 Toolchain and configuration

- Angular CLI 18 (angular.json), configurations: production/demo/client; dev server on 4200 with proxy.conf.json → localhost:1881; bs-config.json for lite serving of dist.
- Dependencies: @angular/material (+cdk), ngx-translate (i18n in assets/i18n/*.json), angular-gridster2 (kiosk/dashboards), angular2-draggable, @ctrl/ngx-codemirror (scripts), svg.js/Raphael/Snap (gauge rendering), chart.js, moment, ngx-matomo style theming, socket.io-client.
- client/dist — production bundle the server serves as the SPA shell.

4.2 Application skeleton — client/src/app/

Folder	Contents
_models/	Typed domain objects: hmi.ts (Hmi/View/Items), device.ts, alarm.ts, apikey.ts, ar.ts (AR markers/cards), chart.ts, graph.ts, project.ts, recipe.ts, report.ts, script.ts, settings.ts, user.ts, plugin.ts, resources.ts, maps.ts, notification.ts, language.ts, client-access.ts
_services/	app.service (Socket.IO client, tag subscriptions/broadcast), hmi.service (viewsTagsSubscribe, onVariableChanged), project.service (project state, view switching, save), settings, user/auth, device, resources, plugin, diagnose, heartbeat, data-converter, rcgi (remote CGI endpoints), toast-notifier, theme, my-file, maps-locations, apikeys, recipe, reports, script, language services
_helpers/, _directives/, _config/	define.ts (huge constant catalog: control types, gauges, enums), utils, svg-utils (widget script processing/ID renaming), flyout/focus directives, config constants
framework/, gui-helpers/, sidenav/, header/, paginator/, iframe/	App frame: top header with save/project menus, side navigation, flyout panels, iframe hosting, pagination helpers
home/, view/, fuxa-view/, cards-view/, lab/	Runtime HMI: Home (view selection, kiosk), fullscreen View renderer, cards-view (gridster dashboards), Lab (view test bench with simulated tag tester — renders the editor's last-selected view)
editor/	The drawing workspace: canvas, drawing tools, element selection/ transform, SVG properties (Interactivity/Property panels with varsToBind tag pickers), onboarding wizard; consumes widget placements via svgWidgetSelected (fetches SVG → localStorage → own_ctrl-image mode)
device/, device-adapter/	Device/tag configuration UIs for all device types incl. browse dialogs, DAQ settings per tag, connection status chips
gauges/	Gauge rendering framework shared by editor and runtime: gauge-base, controls/ (20 control families: value, html-chart, html-graph, html-table, html-image/widget host, html-button/switch/input/select, html-bag, html-iframe, html-video, html-recipe, html-scheduler, panel, pipe (animated pipe), slider, gauge-progress, gauge-semaphore), shapes/ (SVG shape library incl. Proc-Eng symbols), gauge-property/ (flex-variable tag binding, flex-widget-property for widget varsToBind)
alarms/, notifications/	Alarm view, alarm list, ack UIs, notification lists

users/, login/, auth.guard.ts	Sign-in page, user management, role guard for editor routes
resources/	Resource galleries: lib-widgets (server widget library browser with placement selection), kiosk-widgets, image galleries
scripts/, recipes/, reports/, scheduler (in sidenav), apikeys/, plugins/, language/, maps/, logs-view/, diagnose (help)	Feature management pages for their respective subsystems; language editor for i18n texts; maps locations
tester/	Tag value simulator panel used by the Lab (setSignals on mapped gauges)
integrations/node-red/	Node-RED flows management page (when nodeRedEnabled)
ar/	AR mode (this fork): ar-view (fullscreen camera + BarcodeDetector marker detection → live tag cards) and ar-marker-list (marker CRUD backed by project arMarkers table)
auth.guard, material.module, assets/	Route guard, Material module barrel, fonts, i18n, lib (SVG.js etc.), demo project

4.3 Routing map — app.routing.ts (26 routes)

```
'' , /home, /home/:viewName          → HomeComponent            (operator HMI)
/editor                              → EditorComponent          (AuthGuard)
/lab                                 → LabComponent             (AuthGuard)
/device, /rodevice (readonly)        → DeviceComponent          (AuthGuard)
/view, /view/:viewName               → ViewComponent            (fullscreen runtime)
/users, /userRoles                   → Users / UsersRoles       (AuthGuard)
alarms, notifications, scripts, reports, recipes, apikeys,
plugins, mapsLocations, languageTexts, nodeRedFlows, arMarkers  → list/management components
/ar, /ar/:viewName?                 → ArViewComponent          (AR camera mode)
```

4.4 Widget pipeline (client side)

Placement: Lib Widgets panel → `LibWidgetsService.widgetSelected(path)` → editor fetches the SVG, stores the content in **localStorage** keyed by the widget path, and switches to `own_ctrl-image` placement mode. Rendering (html-image component): the stored/saved `svgContent` is parsed, **scripts are extracted and processed** (SvgUtils.processWidget rewrites exported variable hooks and renames element IDs with a GUID to avoid DOM collisions), the sanitized SVG is injected, and the script is executed with a generated module. The 15 exported variables land in `property.varsToBind` where each row binds to device/tag via flex-widget-property. Tag updates arrive through Socket.IO and are pushed into the widget via `putValue(id, value)`; widget interactions call `postValue(id, value)` back to FUXA. Placed widgets **freeze their content at placement time** — new file revisions require re-placement or a new view (version-per-view convention).

5. Data Model and Persistence

Store	Engine	Schema / contents
project.fuxap.db	SQLite (server/_appdata)	general (version), views (id, profile, items incl. placed widget svgContent + varsToBind), devices (id/name/type/property/tags with per-tag daq settings), devicesSecurity, texts, alarms, recipes, notifications, scripts, reports, locations, arMarkers
users.fuxap.db	SQLite	Users with bcrypt hashes and role groups (default admin/123456)
apikeys / alarms / recipes / scheduler .db	SQLite	API keys, alarm history, recipes, schedules
daq-map_<device>.db	SQLite (server/_db)	Tag map: mapid ↔ tag id/name/type per device
daq-data_<device>_<yyyymmddHHMMSS>.db	SQLite (server/_db, archive/)	Time series rows (dt, mapid, value); files tokenized per daqTokenizer hours, old files rotated into archive/ with time-range suffixes, retention cleanup removes archived files
currentTagReadings.db	SQLite	Last value per tag (5 s batched queue) for reconnect restore
External historians	InfluxDB 1.8/2.x, QuestDB, TDengine	Line-protocol writes; selected by settings.daqstore.type; credentials masked in API responses

6. Widget System and the iPTran Schematic

6.1 FUXA dynamic SVG widget standard (docs/HowTo-Widgets.md)

- Pure SVG + embedded `<script>`; bindable globals declared between `//!export-start`/`//!export-end`, named by type prefix: `_ps_` string, `_pn_` number, `_pb_` boolean, `_pc_` color.
- Inbound updates via `putValue(id, value)`; outbound commands via `postValue(id, value)`.
- Widgets using timers must self-destruct when removed (MutationObserver/element check).
- Editor lists widgets from subfolders of _widgets only (GET /api/resources/widgets, admin-gated in secure mode).

6.2 iPTran widget revisions (server/_widgets/dynamicSVG/ and widgets/dynamicSVG/)

File	Revision	Exported vars	Features

iptran_schematic_v1.svg	v1	4 (_ps_)	Static station schematic: Kesinga → Lanjigarh pipeline, PT-01..04 only
iptran_schematic_v2.svg	v2	15	Live flows/level, pump P-101/P-201 states, per-station comm dots, alarm-range coloring (PT 0.5–6 bar, flow 0–2000 m³/h, level 20–90%), click-to-command via postValue, iptran-* ID prefix
iptran_schematic_v3.svg	v3	15 (same names)	Animated pipeline flow (CSS dash animation, speed ∝ flow), blinking alarm banner + SYSTEM NORMAL/N ALARMS chip, 40-sample sparklines per PT, pipeline load bar, live clock + LIVE/STALE (15 s) indicator, station dimming + LINK DOWN on comm loss, pump pulse on command, hover tooltips; 1 s interval with destroy-check
iptran_schematic.svg	latest alias	—	Copy of newest revision for backward compatibility

Deployment mapping (verified in project.fuxap.db): view `iPTran_Overview` (text mock), `iPTran_Schematic` (v2 widget, 13 tag bindings), `iPTran_Live` (v3 widget, same 13 bindings; _cmd variables intentionally unbound). Bound tags: StationA/B/C_PT0x, StationA/B/C_FM*, StationB_LT01, PUMP_A/B_RUN, COMM_ONLINE on the FuxaServer simulation device.

7. Deployment, CI and Desktop Packaging

- **Dockerfile** — three stages: (1) client-builder: node:18-bookworm, npm install, ng build --configuration production; (2) server-builder: build-essential + libsqlite3-dev (+unixodbc via INSTALL_ODBC), npm install, optional node-snap7 (ARG NODE_SNAP); (3) runtime image carrying server + client/dist + node_modules.
- **compose.yml (this deployment)** — image frangoteam/fuxa:latest, restart unless-stopped, five bind mounts (§2.1), port 1881:1881.
- **GitHub Actions** — docker_image.yml / docker_rc.yml / docker_release.yml (image builds and registry push), docs.yml (MkDocs site deploy via pip install mkdocs<2.0 mkdocs-material), electron_latest.yml (desktop builds), headless_packaging.yml (server-only artifacts).
- **Electron (app/electron)** — main.js bootstraps the FUXA server as a child process, settings and project-selection HTML dialogs; used for offline desktop installs.
- **Headless (app/headless)** — headless-entry.js runs the server without the editor for edge/gateway boxes.
- **Documentation site** — MkDocs Material; requirements.txt pins the Python toolchain.

8. Security Model

- Optional `secureEnabled`: JWT access tokens (jsonwebtoken) with configurable expiry, optional HttpOnly refresh-cookie flow, bcrypt password hashes, role groups (admin/operators/guests) checked per API route and for the widget/image galleries.
- Rate limiting: generous API window (1000 req/5 min) and tighter auth window (100/5 min) via express-rate-limit.
- CORS allowlist (settings.allowedOrigins, wildcard patterns supported); BASE_PATH reverse-proxy support.
- Settings disclosure guard: daqstore URLs/credentials stripped from API responses (api/index.js).
- Snapshot route protected by token when security is on; webcam snapshots cleanup by retention.

9. Known Limits and Architectural Roadmap

Findings from the codebase review (full design in the repository notes):

1. **DAQ write path** — single INSERT per tag per poll with drop-on-overload; plan: in-process bounded batching queue (daq-queue.js) + transactional SQLite writes (WAL, prepared statements), preserving source timestamps (device-utils sets tag.timestamp already).

2. **Edge-to-cloud store-and-forward** — no offline buffer today; plan: SQLite spool (WAL ring buffer) + forwarder with exponential backoff, or a Fluent Bit tail adapter over JSON-lines outbox.
3. **Plugin architecture** — drivers are npm-runtime packages but the registry and device-type dispatch are hardcoded (device.js if/else, DaqStoreTypeEnum); plan: IProtocolPlugin/IDBPlugin interfaces + external plugins directory (fuxa.plugin.json manifests) so protocol/DB drivers drop in without core edits.

10. Annotated File Reference

```
server/
├── main.js                    entry point; settings merge; Express/Socket.IO wiring; static
routes
├── fuxa.js                    FUXA facade: init/start/stop, httpApi mount, version
├── paths.js                   path constants
├── envParams.js               env-provided device list (headless)
├── settings.default.js        default settings (copied into _appdata on first run)
├── tsconfig.json, dist/       small TS helper (reports.service.js)
├── project.default.json / project.demo.fuxap    seed/demo projects
├── api/                       REST route groups (auth, projects, daq, alarms, devices/
resources,
│                              users, apikeys, recipes, reports, scheduler, scripts, command,
│                              plugins, diagnose) + jwt-helper, rate limits
├── runtime/
│   ├── index.js               runtime orchestrator; Socket.IO tag broadcast
│   ├── events.js, logger.js, utils.js
│   ├── devices/               device.js state machine + 17 driver folders + device-utils.js
│   ├── storage/               daqstorage.js, sqlite/ (index, currentstorage), influxdb/,
│   │                          questdb/, tdengine/, calculator.js
│   ├── project/ alarms/ users/ apikeys/ notificator/ recipes/ scheduler/ scripts/ jobs/
plugins/
├── integrations/node-red/     Node-RED mount + contrib nodes
├── _widgets/dynamicSVG/       widget source of truth (iPTran v1/v2/v3 + README)
├── _images/                   Pipes/Tanks shape libraries
└── test/                      Mocha suites (api, authorization, devices, storage, plugins,
…)

client/
├── package.json, angular.json, proxy.conf.json, bs-config.json
├── e2e/                       end-to-end tests
├── dist/                      production bundle served by the server
└── src/app/
    ├── app.module.ts, app.routing.ts, app.component.*, material.module.ts, auth.guard.ts
    ├── _models/ _services/ _helpers/ _directives/ _config/
    ├── home/ view/ fuxa-view/ cards-view/ lab/ tester/
    ├── editor/ device/ device-adapter/ gauges/ (controls/, shapes/, gauge-property/, gauge-
base/)
    ├── header/ sidenav/ framework/ gui-helpers/ paginator/ iframe/ help/
    ├── alarms/ notifications/ users/ login/ apikeys/ plugins/
    ├── recipes/ reports/ scripts/ maps/ language/ logs-view/ resources/ (lib-widgets, kiosk-
widgets)
    ├── integrations/node-red/  ar/ (ar-view, ar-marker-list)
    └── assets/ (i18n, fonts, images, lib, project.demo.fuxap)

app/electron/                  desktop packaging (main.js, settings dialogs)
app/headless/                  headless server entry
node-red/node-red-contrib-fuxa Node-RED palette nodes for FUXA
docs/                          ~30 operator HowTo pages + ar-mode.md + settings reference
.github/workflows/             docker_image/rc/release, docs, electron_latest,
headless_packaging
```

Glossary

Term	Meaning
DAQ	Data acquisition — historization of tag values (SQLite/InfluxDB/QuestDB/ TDengine)

Tag	A single named process point (e.g. PT-01 pressure) on a device
View / Item	An SVG canvas in a project; items are placed elements (shapes, gauges, widgets)
varsToBind	Per-placed-widget list of exported SVG variables, each bindable to a device tag
FuxaServer device	Built-in simulation/internal device used for commissioning without PLCs
Lab / Tester	View test bench rendering the editor’s last-selected view with simulated tag values
Widget (dynamic SVG)	SVG + script with _ps_/_pn_/_pb_ exported variables, driven by putValue/ postValue
PlantPAx style	High-performance HMI conventions (gray/limited palette, state colors, alarm ranges) used by the iPTran widget